Relazione di progetto
Gate di Hadamard 2x2 in VHDL su Boolean Board

Versione aggiornata del report

Studente	Vincenzo Crivella

1. Descrizione del progetto e struttura generale

L'obiettivo del progetto è la realizzazione in linguaggio VHDL di un gate di Hadamard 2x2 targettizzato su Boolean Board. Il sistema elabora due campioni di ingresso e produce due uscite secondo la trasformazione di Hadamard: y0 = (x0 + x1) / sqrt(2) e y1 = (x0 - x1) / sqrt(2).

Per consentire la sintesi su FPGA, i dati non sono stati rappresentati con il tipo real ma con una codifica fixed-point signed su 12 bit, di cui 8 dedicati alla parte frazionaria. Il formato adottato è quindi Q4.8. In questo schema, per esempio, 1.0 corrisponde a 256 (0x100), 0.5 corrisponde a 128 (0x080) e -1.0 corrisponde a -256 (0xF00 in complemento a due su 12 bit).

Il coefficiente 1/sqrt(2) è stato approssimato nel formato Q4.8 mediante il valore intero 181. Tale valore deriva da 0.707106 × 2^8 ≈ 181. Durante il calcolo, il circuito esegue la somma e la differenza degli ingressi, quindi moltiplica i risultati per questa costante e infine riallinea il punto fisso con uno shift a destra di 8 bit.

Dal punto di vista strutturale, il progetto è composto da tre file principali di design, da un testbench e dal file combinatorio per l'implementazione su una boolean board.

2. File del progetto

File	Ruolo	Contenuto principale
types_pkg.vhd	Package	Definisce WIDTH=12, FRAC_BITS=8, i tipi signed e il coefficiente HAD_COEFF.
hadamard_core.vhd	Core combinatorio	Calcola somma, differenza, prodotto per il coefficiente e riallineamento Q4.8.
hadamard_board_top.vhd	Top level board	Gestisce switch, segnali di load, selezione uscita, registri e collegamento ai LED.
tb_hadamard_board_top.vhd	Testbench behavioral	Genera clock e reset e applica 4 casi di prova con valori interi e frazionari.
boolean.xdc	Vincoli	Template della Boolean Board

Link. È possibile scaricare i file del progetto dal seguente link:
https://github.com/crivellavincenzo/INTRODUZIONE-AL-TRATTAMENTO-QUANTISTICO-DELLE-INFORMAZIONI

3. Funzionamento della top-level per la Boolean Board

La top-level ***hadamard_board_top*** è stata progettata per adattare il core di calcolo alle risorse fisiche della scheda. Invece di esporre direttamente due ingressi da 12 bit e due uscite da 12 bit, il progetto utilizza un unico bus *sw*(11 downto 0) collegato agli switch per caricare i dati uno alla volta.

Il pulsante *load_x0* carica il valore presente sugli switch nel primo registro di ingresso, mentre *load_x1* carica il secondo operando. Il segnale *sel_out* seleziona quale uscita del gate di Hadamard visualizzare sui *LED*; in questo modo il progetto resta compatibile con la board.

Internamente, ***hadamard_board_top*** istanzia ***hadamard_core***, memorizza gli ingressi in *x_reg*, registra le uscite del core in *y_reg* e infine trasferisce sui LED una delle due uscite selezionate. Questa architettura separa la parte di interfaccia hardware dalla parte strettamente matematica del progetto.

4. Casi di test considerati

Caso	x0	x1	x0 (hex)	x1 (hex)	y0 atteso	y1 atteso
1	1.0	0.0	0x100	0x000	0x0B5	0x0B5
2	1.0	-1.0	0x100	0xF00	0x000	0x16A
3	0.5	1.0	0x080	0x100	0x10F	0xFA5
4	2.0	0.5	0x200	0x080	0x1C4	0x10F

5. Risultato del Linter

Il codice RTL del progetto è stato verificato mediante il tool di Linter integrato in Vivado, con l'obiettivo di individuare eventuali errori di scrittura, segnali inutilizzati, assegnazioni ambigue o costrutti non corretti dal punto di vista RTL.

L'analisi effettuata non ha evidenziato alcuna violazione. In particolare, il report riporta: Violations = 0, Waived = 0 ed esito finale "No Violations found". Questo conferma che i moduli sviluppati risultano corretti dal punto di vista sintattico e strutturale.

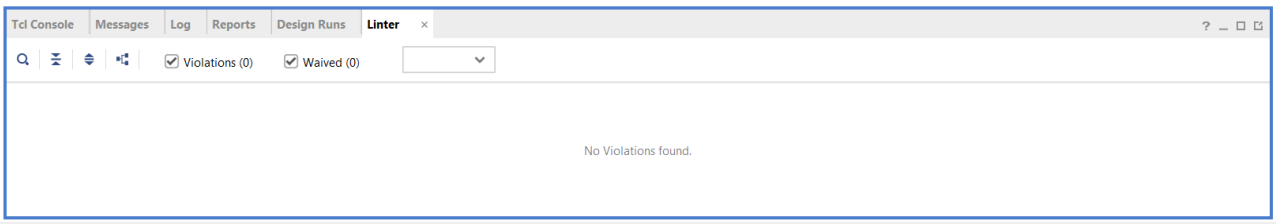

Figura 1 – Risultato del Linter: nessuna violazione rilevata.

6. Risultati di implementazione

Dopo la fase di sintesi e implementazione, il progetto è stato analizzato mediante i report generati da Vivado, in modo da valutarne il comportamento sulla FPGA della Boolean Board. I principali aspetti considerati sono stati l'utilizzo delle risorse FPGA, prestazioni temporali del circuito e dissipazione di potenza.

6.1 Utilizzo delle risorse FPGA

Il report di implementazione mostra che il progetto utilizza una quantità molto contenuta di risorse logiche, coerente con la ridotta complessità del gate di Hadamard 2x2.

I valori ottenuti sono i seguenti: **LUT** = 13 su 32600 (0.04%), **FF** = 37 su 65200 (0.06%), **DSP** = 2 su 120 (1.67%), **IO** = 29 su 210 (13.81%), **BUFG** = 1 su 32 (3.13%). L'utilizzo di 2 DSP è coerente con le moltiplicazioni presenti nel modulo hadamard_core, mentre l'occupazione delle risorse di I/O è più alta a causa del collegamento diretto della top-level con switch, pulsanti, LED, clock e reset della board.

Utilization **Post-Synthesis** | Post-Implementation

Graph | **Table**

Resource	Estimation	Available	Utilization %
LUT	13	32600	0.04
FF	37	65200	0.06
DSP	2	120	1.67
IO	29	210	13.81
BUFG	1	32	3.13

Power **Summary** | On-Chip

Total On-Chip Power:	**0.111 W**
Junction Temperature:	**25,5 °C**
Thermal Margin:	59,5 °C (12,0 W)
Effective ϑJA:	4,9 °C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Implemented Power Report

Figura 2 – Utilizzo delle risorse FPGA e riepilogo della potenza

6.2 Analisi temporale e frequenza massima di clock

Dal report di timing post-implementation risultano i seguenti parametri: Worst Negative Slack (WNS) = 3.06 ns, Total Negative Slack (TNS) = 0 ns, Number of Failing Endpoints = 0, Total Number of Endpoints = 122.

Il vincolo di clock impostato nel file XDC è pari a 10 ns, corrispondente a una frequenza di 100 MHz. Il valore positivo del WNS indica che il progetto soddisfa correttamente il vincolo temporale. Stimando il periodo critico come T_critico ≈ 10 ns - 3.06 ns = 6.94 ns, si ottiene una frequenza massima teorica pari a circa 144 MHz. Il progetto risulta quindi pienamente compatibile con il clock a 100 MHz della Boolean Board.

Timing **Setup** | Hold | Pulse Width

Worst Negative Slack (WNS):	3.06 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	122

Implemented Timing Report

Figura 3 – Timing report

6.3 Dissipazione di potenza

Il report di potenza riporta i seguenti valori principali: *Total On-Chip Power* = 0.111 W, *Junction Temperature* = 25.5 °C, *Thermal Margin* = 59.5 °C, *Effective ϑJA* = 4.9 °C/W, *Confidence Level* = Low.

La potenza totale risulta molto contenuta. Dal dettaglio della potenza si osserva una componente dinamica pari a 0.039 W (35%) e una componente statica pari a 0.072 W (65%). Tra i contributi dinamici, la parte predominante è dovuta alle I/O, con circa 0.035 W, mentre clocks, signals e logic incidono in misura minima. Il confidence level basso indica che la stima di potenza è preliminare e non basata su un file dettagliato di switching activity.

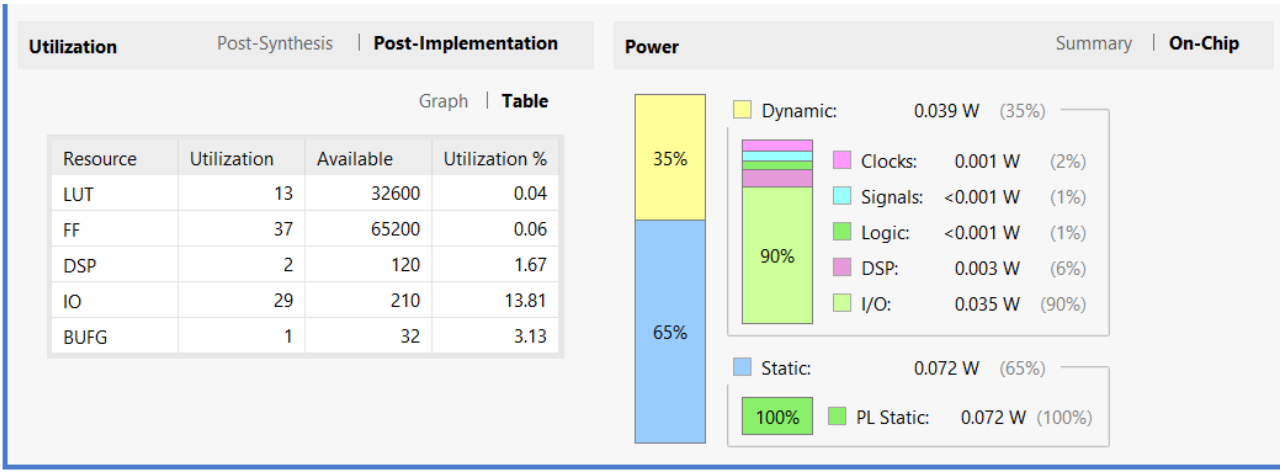

Figura 4 – Report di implementazione: utilizzo risorse, timing e potenza.

6.4 Descrizione più dettagliata dell'implementazione

L'implementazione del progetto è stata organizzata in maniera modulare, separando la definizione dei tipi, il nucleo computazionale e l'interfaccia fisica verso la Boolean Board.

- Nel file *types_pkg.vhd* sono definite le costanti WIDTH = 12 e FRAC_BITS = 8, i tipi signed utilizzati nel progetto e la costante HAD_COEFF = 181, che rappresenta l'approssimazione di 1/sqrt(2) nel formato Q4.8.
- Il modulo *hadamard_core.vhd* realizza il nucleo combinatorio del gate di Hadamard. Esso riceve due campioni in ingresso, calcola somma e differenza, moltiplica entrambi i risultati per HAD_COEFF e infine riallinea il formato fixed-point mediante uno shift a destra di 8 bit.
- Il file *hadamard_board_top.vhd* costituisce la top-level finale del progetto. Il bus sw(11 downto 0) viene usato per inserire un dato alla volta tramite gli switch della board.
  - Quando load_x0 = '1', il valore presente su sw viene caricato nel primo registro di ingresso x_reg(0).
  - Quando load_x1 = '1', viene caricato il secondo registro x_reg(1).

  Il modulo hadamard_core riceve i due valori registrati e produce due uscite, che vengono memorizzate in y_reg.

  In base al valore di sel_out, uno dei due risultati viene trasferito su led_reg e mostrato sui LED della board.

- Il testbench *tb_hadamard_board_top.vhd* genera il clock di simulazione, il reset iniziale, i valori applicati al bus sw, gli impulsi di load_x0 e load_x1 e il segnale sel_out. I quattro casi di prova utilizzati includono valori positivi, negativi e frazionari, confermando la corretta gestione del formato fixed-point Q4.8 e il corretto funzionamento complessivo del sistema.

7. Risultati ottenuti: schematic del design

Lo schematic di implementazione conferma la presenza delle porte top-level clk, rst, sw[11:0], load_x0, load_x1, sel_out e led[11:0], oltre alla presenza del blocco centrale had_inst: hadamard_core. Si osservano inoltre i registri di ingresso x_reg, i registri di uscita y_reg e i registri che pilotano i LED, coerentemente con la struttura progettuale prevista.

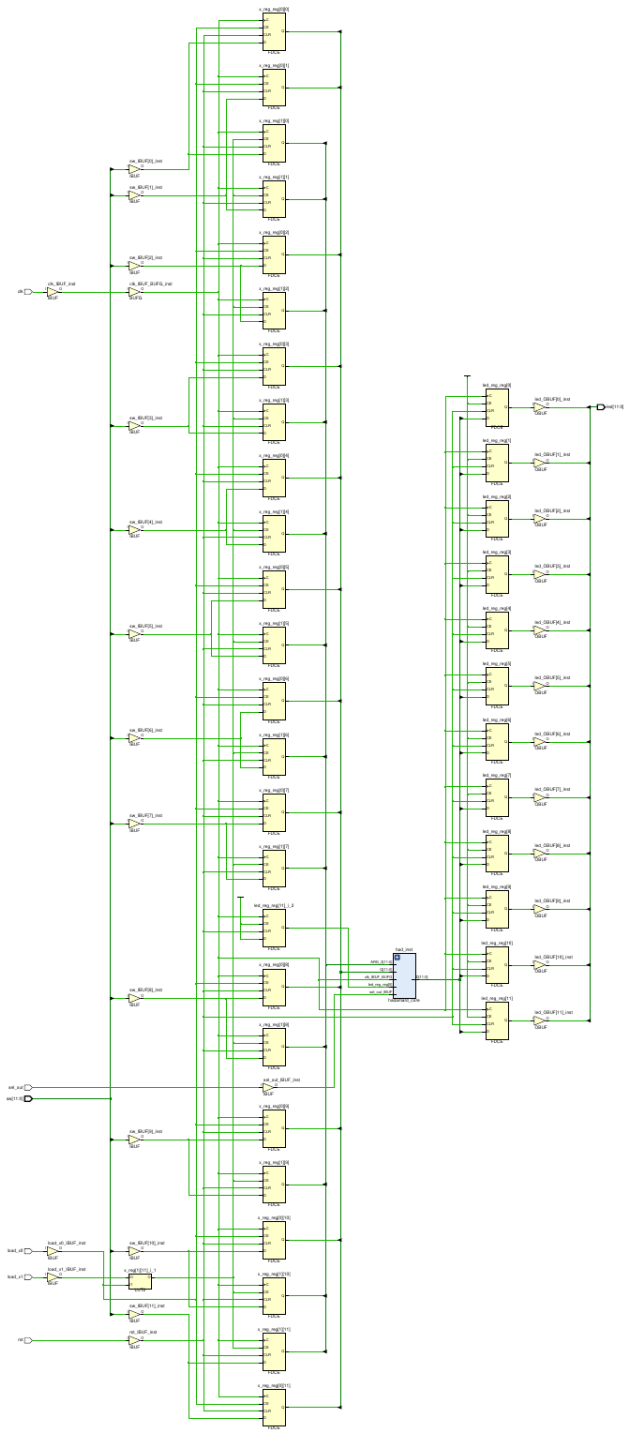

Figura 5 – Schematic del design sintetizzato per la top-level hadamard_board_top.

La versione precedente è stata ottenuta dall'implementazione del progetto, mentre la seguente immagine mostra ciò che è stato ottenuto dopo la Run Linter.

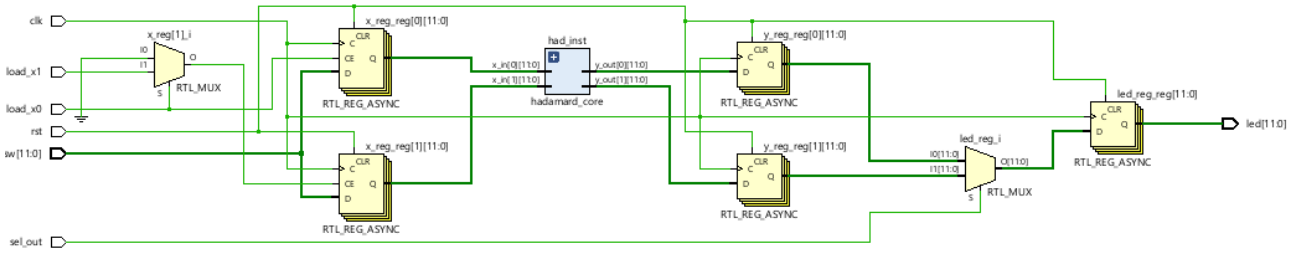

Figura 6 – Schematic del Design ottenuta dopo il run Linter

7.1 Device ottenuto dall'implementazione

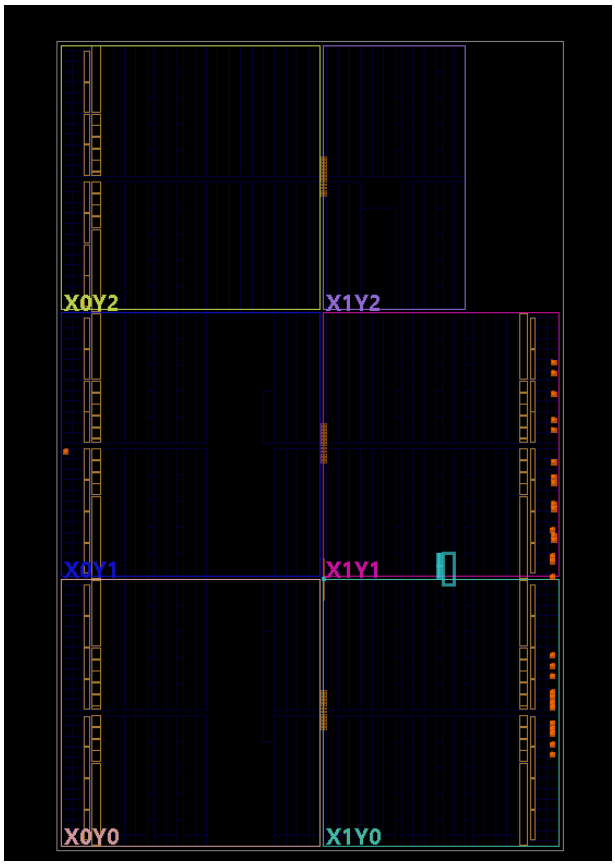

Figura 7 – Devide della Boolean board generato

8. Risultati ottenuti: simulazione behavioral

Le immagini seguenti riportano le forme d'onda ottenute durante la simulazione behavioral del testbench tb_hadamard_board_top. I quattro screenshot mostrano l'evoluzione del clock, dei segnali di caricamento, del bus sw e dei valori osservati sui LED durante i quattro casi di test.

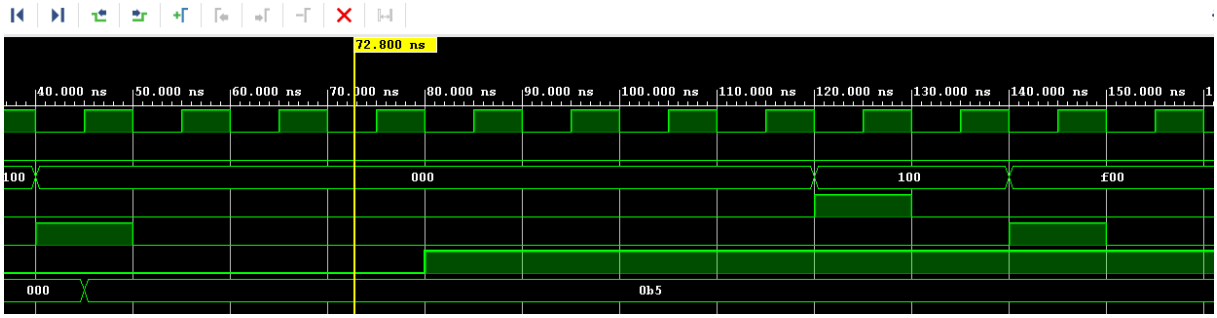

Figura 8 – Primo tratto della simulazione: caricamento del caso 1 e stabilizzazione dell'uscita 0x0B5.

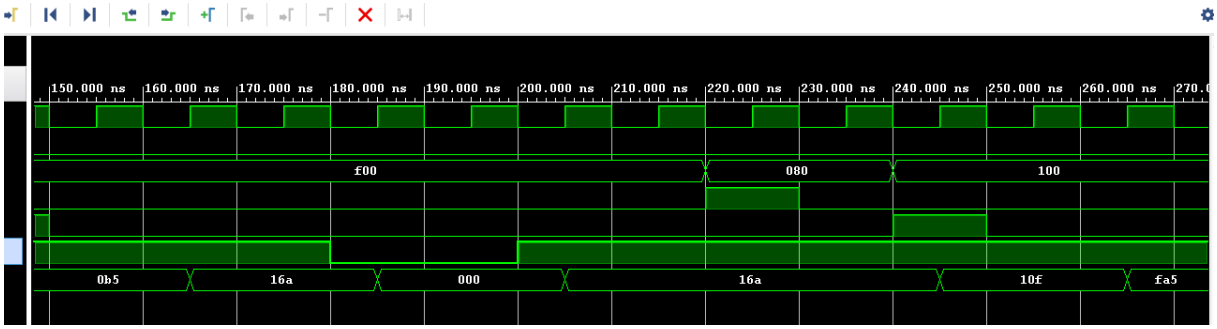

Figura 9 – Secondo tratto della simulazione: completamento del caso 2 e avvio del caso 3.

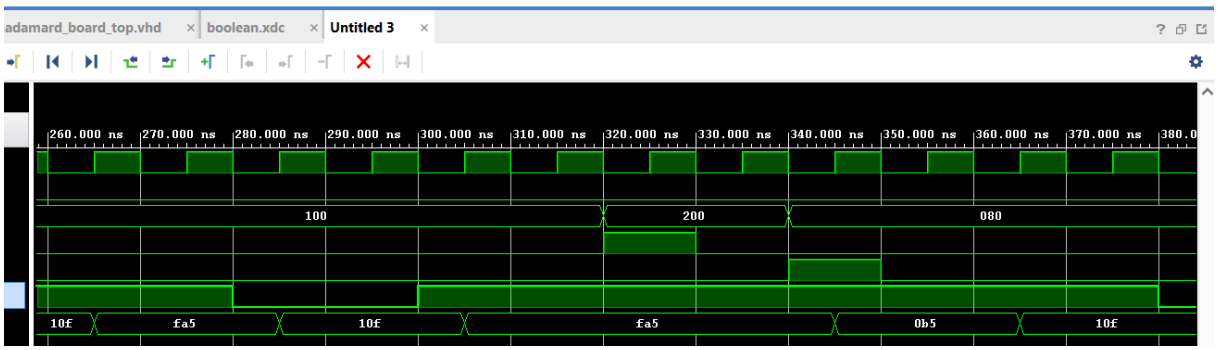

Figura 10 – Terzo tratto della simulazione: osservazione dei risultati del caso 3 e caricamento del caso 4.

Figura 11 – Quarto tratto della simulazione: valori finali del caso 4.

9. Codice sorgente

Di seguito è riportato il codice dei file principali del progetto, incluso il testbench e il file dei vincoli usato come base per la Boolean Board.

types_pkg.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

package types_pkg is
    -- definizioni costanti

    -- formato dati: 12 bit totali, 8 bit frazionari
    constant WIDTH      : integer := 12;
    constant FRAC_BITS  : integer := 8; -- parte frazionaria
    constant VEC_SIZE   : integer := 2;

    -- tipo base dato: di 12 bit, di cui 8 frazionari - quindi sarà 11 downto 0
    subtype data_t is signed(WIDTH-1 downto 0);

    -- vettore a 2 elementi - cioè 0 to 1
    type vec2_t is array (0 to VEC_SIZE-1) of data_t;

    -- somma e prodotto
    constant SUM_WIDTH  : integer := WIDTH + 1; -- per somma/differenza può servire un bit in più
    constant PROD_WIDTH : integer := SUM_WIDTH + WIDTH; -- per moltiplicazione
    subtype sum_t  is signed(SUM_WIDTH-1 downto 0);
    subtype prod_t is signed(PROD_WIDTH-1 downto 0);

    --had coeff

    -- partendo da 1/sqrt(2) e 256 (2^8), 8 indicano gli 8 bit, calcoliamo had coeff (sarà 181)
    -- coefficiente 1/sqrt(2) in formato Q4.8
    -- 1/sqrt(2) ≈ 0.707106...
    -- 0.707106 * 2^8 ≈ 181
    constant HAD_COEFF_INT : integer := 181;
    constant HAD_COEFF : data_t := to_signed(HAD_COEFF_INT, WIDTH);

end package types_pkg;
```

hadamard_core.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.types_pkg.ALL;

entity hadamard_core is
    Port (
        x_in  : in  vec2_t;
        y_out : out vec2_t
    );
end hadamard_core;

architecture Behavioral of hadamard_core is
begin

    had_prod: process(x_in)
        variable sum_v  : sum_t;
        variable diff_v : sum_t;
        variable prod0_v : prod_t;
        variable prod1_v : prod_t;
    begin
        -- somma e differenza degli ingressi
        sum_v  := resize(x_in(0), SUM_WIDTH) + resize(x_in(1), SUM_WIDTH);
        diff_v := resize(x_in(0), SUM_WIDTH) - resize(x_in(1), SUM_WIDTH);

        -- moltiplicazione per 1/sqrt(2) in formato fixed-point
        prod0_v := sum_v  * HAD_COEFF;
        prod1_v := diff_v * HAD_COEFF;

        -- riallineamento al formato Q4.8:
        -- dopo la moltiplicazione ci sono 16 bit frazionari,
        -- quindi facciamo shift right di 8 per tornare a 8 bit frazionari
        y_out(0) <= resize(shift_right(prod0_v, FRAC_BITS), WIDTH);
        y_out(1) <= resize(shift_right(prod1_v, FRAC_BITS), WIDTH);
    end process;

end Behavioral;
```

hadamard_board_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.types_pkg.ALL;

entity hadamard_board_top is
    Port (
        clk, rst                    : in  STD_LOGIC;
        sw                          : in  std_logic_vector(WIDTH-1 downto 0);
        load_x0, load_x1, sel_out   : in  STD_LOGIC;
        led                         : out std_logic_vector(WIDTH-1 downto 0)
    );
end hadamard_board_top;

architecture Structural of hadamard_board_top is

    component hadamard_core is
        Port (
            x_in  : in  vec2_t;
            y_out : out vec2_t
        );
    end component;

    signal x_reg   : vec2_t := (others => (others => '0'));
    signal y_comb  : vec2_t;
    signal y_reg   : vec2_t := (others => (others => '0'));
    signal led_reg : std_logic_vector(WIDTH-1 downto 0) := (others => '0');

begin
    had_inst : hadamard_core
        port map (
            x_in  => x_reg,
            y_out => y_comb
        );

    reg_proc : process(clk, rst)
    begin
        if rst = '1' then
            x_reg   <= (others => (others => '0'));
            y_reg   <= (others => (others => '0'));
            led_reg <= (others => '0');

        elsif rising_edge(clk) then

            -- caricamento dei due ingressi dai 12 switch
            -- se premi load_x0, carichiamo il primo valore

            if load_x0 = '1' then
                x_reg(0) <= signed(sw);
            elsif load_x1 = '1' then
                x_reg(1) <= signed(sw);
            end if;

            -- registrazione delle uscite del core
            y_reg <= y_comb;

            -- visualizzazione registrata di una sola uscita alla volta
            if sel_out = '0' then
                led_reg <= std_logic_vector(y_reg(0));
            else
                led_reg <= std_logic_vector(y_reg(1));
            end if;
        end if;
    end process;
    led <= led_reg;

end Structural;
```

tb_hadamard_board_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.types_pkg.ALL;

entity tb_hadamard_board_top is
end tb_hadamard_board_top;

architecture Behavioral of tb_hadamard_board_top is

    signal clk     : std_logic := '0';
    signal rst     : std_logic := '0';
    signal sw      : std_logic_vector(WIDTH-1 downto 0) := (others => '0');
    signal load_x0 : std_logic := '0';
    signal load_x1 : std_logic := '0';
    signal sel_out : std_logic := '0';
    signal led     : std_logic_vector(WIDTH-1 downto 0);

begin

    DUT : entity work.hadamard_board_top
        port map (
            clk     => clk,
            rst     => rst,
            sw      => sw,
            load_x0 => load_x0,
            load_x1 => load_x1,
            sel_out => sel_out,
            led     => led
        );

    clk_process : process
    begin
        clk <= '0';
        wait for 5 ns;
        clk <= '1';
        wait for 5 ns;
    end process;

    rst <= '1', '0' after 12 ns;

    -- Casi:
    -- 1) x0=1.0   x1=0.0
    -- 2) x0=1.0   x1=-1.0
    -- 3) x0=0.5   x1=1.0
    -- 4) x0=2.0   x1=0.5

    sw <= std_logic_vector(to_signed(16#100#, WIDTH)),
          std_logic_vector(to_signed(16#000#, WIDTH)) after 40 ns,
          std_logic_vector(to_signed(16#100#, WIDTH)) after 120 ns,
          std_logic_vector(to_signed(-256, WIDTH))    after 140 ns,
          std_logic_vector(to_signed(16#080#, WIDTH)) after 220 ns,
          std_logic_vector(to_signed(16#100#, WIDTH)) after 240 ns,
          std_logic_vector(to_signed(16#200#, WIDTH)) after 320 ns,
          std_logic_vector(to_signed(16#080#, WIDTH)) after 340 ns;

    load_x0 <= '0',
               '1' after 20 ns,  '0' after 30 ns,
               '1' after 120 ns, '0' after 130 ns,
               '1' after 220 ns, '0' after 230 ns,
               '1' after 320 ns, '0' after 330 ns;

    load_x1 <= '0',
               '1' after 40 ns,  '0' after 50 ns,
               '1' after 140 ns, '0' after 150 ns,
               '1' after 240 ns, '0' after 250 ns,
               '1' after 340 ns, '0' after 350 ns;
```

```
    -- mostro prima y0 e poi y1 per ciascun caso
    sel_out <= '0',
               '1' after 80 ns,   -- caso 1: y1
               '0' after 180 ns,  -- caso 2: y0
               '1' after 200 ns,  -- caso 2: y1
               '0' after 280 ns,  -- caso 3: y0
               '1' after 300 ns,  -- caso 3: y1
               '0' after 380 ns,  -- caso 4: y0
               '1' after 400 ns;  -- caso 4: y1

end Behavioral;
```

boolean.xdc

```
# clk input is from the 100 MHz oscillator on Boolean board
create_clock -period 10.000 -name gclk [get_ports clk]
set_property -dict {PACKAGE_PIN F14 IOSTANDARD LVCMOS33} [get_ports {clk}]

# Set Bank 0 voltage
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

# On-board Slide Switches
set_property -dict {PACKAGE_PIN V2 IOSTANDARD LVCMOS33} [get_ports {sw[0]}]
set_property -dict {PACKAGE_PIN U2 IOSTANDARD LVCMOS33} [get_ports {sw[1]}]
set_property -dict {PACKAGE_PIN U1 IOSTANDARD LVCMOS33} [get_ports {sw[2]}]
set_property -dict {PACKAGE_PIN T2 IOSTANDARD LVCMOS33} [get_ports {sw[3]}]
set_property -dict {PACKAGE_PIN T1 IOSTANDARD LVCMOS33} [get_ports {sw[4]}]
set_property -dict {PACKAGE_PIN R2 IOSTANDARD LVCMOS33} [get_ports {sw[5]}]
set_property -dict {PACKAGE_PIN R1 IOSTANDARD LVCMOS33} [get_ports {sw[6]}]
set_property -dict {PACKAGE_PIN P2 IOSTANDARD LVCMOS33} [get_ports {sw[7]}]
set_property -dict {PACKAGE_PIN P1 IOSTANDARD LVCMOS33} [get_ports {sw[8]}]
set_property -dict {PACKAGE_PIN N2 IOSTANDARD LVCMOS33} [get_ports {sw[9]}]
set_property -dict {PACKAGE_PIN N1 IOSTANDARD LVCMOS33} [get_ports {sw[10]}]
set_property -dict {PACKAGE_PIN M2 IOSTANDARD LVCMOS33} [get_ports {sw[11]}]

# On-board LEDs
set_property -dict {PACKAGE_PIN G1 IOSTANDARD LVCMOS33} [get_ports {led[0]}]
set_property -dict {PACKAGE_PIN G2 IOSTANDARD LVCMOS33} [get_ports {led[1]}]
set_property -dict {PACKAGE_PIN F1 IOSTANDARD LVCMOS33} [get_ports {led[2]}]
set_property -dict {PACKAGE_PIN F2 IOSTANDARD LVCMOS33} [get_ports {led[3]}]
set_property -dict {PACKAGE_PIN E1 IOSTANDARD LVCMOS33} [get_ports {led[4]}]
set_property -dict {PACKAGE_PIN E2 IOSTANDARD LVCMOS33} [get_ports {led[5]}]
set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33} [get_ports {led[6]}]
set_property -dict {PACKAGE_PIN E5 IOSTANDARD LVCMOS33} [get_ports {led[7]}]
set_property -dict {PACKAGE_PIN E6 IOSTANDARD LVCMOS33} [get_ports {led[8]}]
set_property -dict {PACKAGE_PIN C3 IOSTANDARD LVCMOS33} [get_ports {led[9]}]
set_property -dict {PACKAGE_PIN B2 IOSTANDARD LVCMOS33} [get_ports {led[10]}]
set_property -dict {PACKAGE_PIN A2 IOSTANDARD LVCMOS33} [get_ports {led[11]}]

# On-board Buttons
set_property -dict {PACKAGE_PIN J2 IOSTANDARD LVCMOS33} [get_ports {load_x0}]
set_property -dict {PACKAGE_PIN J5 IOSTANDARD LVCMOS33} [get_ports {load_x1}]
set_property -dict {PACKAGE_PIN H2 IOSTANDARD LVCMOS33} [get_ports {sel_out}]
set_property -dict {PACKAGE_PIN J1 IOSTANDARD LVCMOS33} [get_ports {rst}]
```